UNIT I

Principles of Object-Oriented Programming & Control Structures

ANSWERED QUESTION BANK

Debugging • Pseudo-Code • Dry Runs • Outputs • Quick Revision

Section A – Debugging Questions

Question 1: Operator Precedence and Unsigned Loop

Topic: Bitwise operators, precedence, implicit promotion, unsigned data types.

Errors and explanations

- Line 78 – Logic defect: == has higher precedence than bitwise &: num & 1 == 1 is parsed as num & (1 == 1), which becomes num & 1. This may appear to work in a condition but does not express the intended comparison clearly; use (num & 1) == 1.
- Line 85 – Loop-boundary defect: unsigned char cannot represent -1. When i is 0 and decremented, it wraps to its maximum value (typically 255), so i >= 0 remains true. The temporary break prevents this wraparound; a signed int counter is the robust solution.

Corrected complete C++ code

```
#include <iostream>

int main() {
    unsigned char num = 15;

    if (((num & 1) == 1) && (num > 10)) {
        std::cout << "Valid Odd Number: " << static_cast<int>(num) << std::endl;
    } else {
        std::cout << "Condition Failed for: " << static_cast<int>(num) << std::endl;
    }

    for (int i = 3; i >= 0; --i) {
        std::cout << i << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

Dry run

Step	num	Odd test	Greater than 10	Result / output
1	15	(15 & 1) == 1 → true	15 > 10 → true	Print Valid Odd Number: 15
2	i = 3	—	3 >= 0	Print 3
3	i = 2	—	2 >= 0	Print 2
4	i = 1	—	1 >= 0	Print 1

5	i = 0	—	0 >= 0	Print 0; then i becomes -1 and loop stops

Exact output:

```
Valid Odd Number: 15
3 2 1 0
```

Question 2: Scope Resolution and Variable Shadowing

Errors and explanations

- Line 118 – Compilation error: :taxRate is invalid syntax. The global variable is accessed with the scope resolution operator as ::taxRate.
- Line 124 – Semantic error: ::taxRate refers to the global variable (0.18), not the outer local variable (0.05). The unary scope operator cannot access an enclosing function-local variable hidden by a nested local variable. Give local variables distinct names and calculate the outer-local total before entering the inner block.

Corrected complete C++ code

```
#include <iostream>

double taxRate = 0.18; // Global tax rate

int main() {
   double itemPrice = 1000.0;
   double subsidizedRate = 0.05;

   double totalWithLocalTax = itemPrice + itemPrice * subsidizedRate;
   double totalWithGlobalTax = itemPrice + itemPrice * ::taxRate;

   std::cout << "Local Rate: " << subsidizedRate << std::endl;
   {
      int corporateTaxPercent = 12;
      double innerTotal = itemPrice + itemPrice * corporateTaxPercent / 100.0;
      std::cout << "Inner total: " << innerTotal << std::endl;
   }

   std::cout << "Final Global Calculation: " << totalWithGlobalTax << std::endl;
   std::cout << "Final Local Calculation: " << totalWithLocalTax << std::endl;
   return 0;
}
```

Dry run

Calculation	Working	Result
Local tax	1000 + (1000 × 0.05)	1050
Inner corporate tax	1000 + (1000 × 12 / 100)	1120
Global tax	1000 + (1000 × 0.18)	1180

Exact output:

```
Local Rate: 0.05
Inner total: 1120
Final Global Calculation: 1180
Final Local Calculation: 1050
```

Question 3: Switch-Case Initialization and Illegal Case Type

Errors and explanations

- Line 158 – Compilation error: declaring and initializing multiplier directly under a case label can allow control to jump past its initialization from another case. Put each case body inside braces to create its own scope.
- Line 166 – Compilation error: a switch expression must have an integral or enumeration type; case labels must be integral constant expressions. A floating-point label such as 4.5 is not permitted.
- Line 160 – Logic defect: case 2 has no break, so execution falls through into case 3 and adds 100 unintentionally.

Corrected complete C++ code

```
#include <iostream>

int main() {
    int tokenType = 2;
    float tokenValue = 0.0f;

    switch (tokenType) {
        case 1: {
            int offset = 10;
            tokenValue = 50.0f + offset;
            break;
        }
        case 2: {
            int multiplier = 5;
            tokenValue = 20.0f * multiplier;
            break;
        }
        case 3: {
            tokenValue += 100.0f;
            break;
        }
        case 4: {
            std::cout << "Integer token category 4 detected" << std::endl;
            break;
        }
        default: {
            std::cout << "Unknown token" << std::endl;
        }
    }

    std::cout << "Calculated Value: " << tokenValue << std::endl;
    return 0;
}
```

Note: Case 4 is used as an integer category in place of the invalid floating-point case 4.5. If the application needs fractional token values, test them with if/else rather than switch.

Dry run

Step	Execution	tokenValue
1	tokenType = 2; enter case 2	0.0
2	multiplier = 5	0.0
3	20.0 × 5	100.0
4	break exits switch	100.0

Exact output:

```
Calculated Value: 100
```

Question 4: Reference Initialization and Constant Binding

Errors and explanations

- Line 197 – Compilation error: a reference must be initialized when it is declared. Use int& refA = a.
- Line 201 – Compilation error: a non-const lvalue reference cannot bind to a temporary literal. Use const int& for a read-only reference, or create a named integer.
- Line 205 – Compilation error: int& cannot bind to the temporary int produced by converting the double pi. Create a real int variable using an explicit cast, then bind the reference to it.
- Line 209 – Compilation error: constRef is read-only and cannot be assigned to.

Lifetime extension: when a const lvalue reference binds directly to a temporary object, the temporary remains alive for the lifetime of that reference (within the relevant scope). The reference does not make the temporary modifiable.

Corrected complete C++ code

```
#include <iostream>

int main() {
    int a = 25;
    int& refA = a;

    const int& refLiteral = 100;

    double pi = 3.14159;
    int convertedPi = static_cast<int>(pi);
    int& refInt = convertedPi;

    const int& constRef = 50;

    std::cout << "a and refA: " << a << " " << refA << std::endl;
    std::cout << "Literal reference: " << refLiteral << std::endl;
    std::cout << "Converted reference: " << refInt << std::endl;
    std::cout << "Const reference: " << constRef << std::endl;
    return 0;
}
```

Dry run

Variable / reference	Value	Explanation
a / refA	25 / 25	refA aliases a
refLiteral	100	Const reference binds to temporary; lifetime is extended
pi	3.14159	Original double value
convertedPi / refInt	3 / 3	Explicit conversion truncates fractional part; refInt aliases convertedPi
constRef	50	Read-only reference; no assignment is attempted

Exact output:

```
a and refA: 25 25
Literal reference: 100
Converted reference: 3
Const reference: 50
```

Question 5: Dynamic Array Allocation and Stray Semicolon

Errors and explanations

- Line 237 – Logic/compilation defect: the semicolon immediately after the for statement makes the loop body empty. The following block is separate, and i is out of scope there because it was declared in the for initializer.
- Line 249 – Undefined behavior: memory allocated with new[] must be released with delete[]. Using scalar delete for an array is a mismatched deallocation.

Corrected complete C++ code

```
#include <iostream>

int main() {
    int size = 5;
    int* readings = new int[size];

    for (int i = 0; i < size; ++i) {
        readings[i] = (i + 1) * 10;
    }

    int total = 0;
    for (int i = 0; i < size; ++i) {
        total += readings[i];
    }

    std::cout << "Total: " << total << std::endl;

    delete[] readings;
    return 0;
}
```

Dry run

i	Assigned reading	Running total
0	10	10
1	20	30
2	30	60
3	40	100
4	50	150

Exact output:

```
Total: 150
```

Memory rule: use delete for one object created by new, and delete[] for an array created by new[].

Section B – Pseudo-Code Writing Questions

Question 1: Procedural-to-OOP Paradigm Transformation

Design overview

- WaterConsumer is a class that groups one consumer's readings and bill data.
- Private attributes protect the data from direct outside modification (encapsulation/data hiding).
- Initialize validates the readings; CalculateConsumption computes the difference; ComputeBill applies progressive tariff tiers; DisplayBill prints the invoice.
- For the supplied readings, consumption is 370 units. Since usage is above 300 units, apply the 10% surcharge to the complete tiered bill.

Structured pseudo-code

```
CLASS WaterConsumer
PRIVATE:
  consumerID : STRING
  previousReading : REAL
  currentReading : REAL
  unitsConsumed : REAL
  totalBill : REAL
  valid : BOOLEAN

PUBLIC:
  CONSTRUCTOR Initialize(id : STRING, prev : REAL, curr : REAL)
    consumerID ← id
    valid ← TRUE
    IF prev < 0 OR curr < 0 OR curr < prev THEN
      valid ← FALSE
      DISPLAY "ERROR: Invalid meter readings"
    ELSE
      previousReading ← prev
      currentReading ← curr
      unitsConsumed ← 0
      totalBill ← 0
    ENDIF
  END CONSTRUCTOR

  METHOD CalculateConsumption()
    IF valid = TRUE THEN
      unitsConsumed ← currentReading - previousReading
    ENDIF
  END METHOD

  METHOD ComputeBill()
    DECLARE firstTier, secondTier, thirdTier, surcharge : REAL
    IF valid = TRUE THEN
      firstTier ← MIN(unitsConsumed, 100) * 1.50
      secondTier ← MIN(MAX(unitsConsumed - 100, 0), 200) * 2.50
      thirdTier ← MAX(unitsConsumed - 300, 0) * 4.00
      totalBill ← firstTier + secondTier + thirdTier

      IF unitsConsumed > 300 THEN
        surcharge ← totalBill * 0.10
        totalBill ← totalBill + surcharge
      ENDIF
    ENDIF
  END METHOD

  METHOD DisplayBill()
```

```
        IF valid = TRUE THEN
            DISPLAY "Consumer ID: ", consumerID
            DISPLAY "Units Consumed: ", unitsConsumed
            DISPLAY "Total Bill: $", totalBill
        ENDIF
    END METHOD
ENDCLASS

BEGIN MAIN
    DECLARE consumer : WaterConsumer
    consumer.Initialize("WC101", 1450.0, 1820.0)
    consumer.CalculateConsumption()
    consumer.ComputeBill()
    consumer.DisplayBill()
END MAIN
```

Dry-run trace

Stage	Calculation	Amount / value
Input	Previous = 1450.0; Current = 1820.0	—
Consumption	1820.0 − 1450.0	370 units
First 100 units	100 × $1.50	$150.00
Next 200 units	200 × $2.50	$500.00
Above 300 units	70 × $4.00	$280.00
Subtotal	150 + 500 + 280	$930.00
Surcharge	10% × 930	$93.00
Final bill	930 + 93	$1,023.00

Expected invoice values: Consumer ID WC101; Units Consumed 370; Total Bill $1,023.00.

Question 2: Multi-Tier Tax and Precision Cast Engine

Design overview

- Cap each eligible deduction at its statutory limit before subtracting it from gross salary.
- Clamp taxable income to zero if eligible deductions exceed gross income.
- Apply progressive tax only to the income portion in each bracket; add 4% cess to raw tax.
- Round to the nearest whole rupee using INTEGER conversion of tax + 0.5.

Structured pseudo-code

```
CLASS TaxCalculationEngine
PRIVATE:
    grossAnnualSalary : REAL
    deduction80C : REAL
    deductionHealthInsurance : REAL
    eligible80C : REAL
    eligibleHealth : REAL
    taxableIncome : REAL
    rawTax : REAL
    cess : REAL
    finalTax : REAL
    roundedTax : INTEGER

PUBLIC:
    METHOD CalculateTax(gross : REAL, d80 : REAL, health : REAL)
        grossAnnualSalary ← gross
        eligible80C ← MIN(MAX(d80, 0), 150000)
        eligibleHealth ← MIN(MAX(health, 0), 25000)

        taxableIncome ← grossAnnualSalary - eligible80C - eligibleHealth
```

```
        IF taxableIncome < 0 THEN
            taxableIncome ← 0
        ENDIF

        rawTax ← 0
        IF taxableIncome > 250000 THEN
            rawTax ← rawTax + MIN(taxableIncome - 250000, 250000) * 0.05
        ENDIF
        IF taxableIncome > 500000 THEN
            rawTax ← rawTax + MIN(taxableIncome - 500000, 500000) * 0.20
        ENDIF
        IF taxableIncome > 1000000 THEN
            rawTax ← rawTax + (taxableIncome - 1000000) * 0.30
        ENDIF

        cess ← rawTax * 0.04
        finalTax ← rawTax + cess
        roundedTax ← INTEGER(finalTax + 0.5)
        DISPLAY "Taxable Income: ", taxableIncome
        DISPLAY "Raw Tax: ", rawTax
        DISPLAY "Cess: ", cess
        DISPLAY "Rounded Tax: ", roundedTax
    END METHOD
ENDCLASS

BEGIN MAIN
    DECLARE engine : TaxCalculationEngine
    engine.CalculateTax(1200000, 180000, 30000)
END MAIN
```

Dry-run trace

Stage	Calculation	Result
80C eligible deduction	MIN(180,000, 150,000)	$150,000
Health eligible deduction	MIN(30,000, 25,000)	$25,000
Taxable income	1,200,000 − 150,000 − 25,000	$1,025,000
0% bracket	First $250,000	$0
5% bracket	250,000 × 0.05	$12,500
20% bracket	500,000 × 0.20	$100,000
30% bracket	25,000 × 0.30	$7,500
Raw tax	12,500 + 100,000 + 7,500	$120,000
Cess	120,000 × 0.04	$4,800
Tax including cess	120,000 + 4,800	$124,800
Rounded integer	INTEGER(124,800 + 0.5)	$124,800

Final result: taxable income = $1,025,000; rounded tax including cess = $124,800.

Question 3: Bitwise Mask Validation and Sensor Health Telemetry

Required hexadecimal masks

Name	Hex mask	Meaning
GYROSCOPE	0x01	Bit 0 must be 1
ACCELEROMETER	0x02	Bit 1 must be 1
BAROMETER	0x04	Bit 2 must be 1
GPS	0x08	Bit 3 must be 1
LOW_BATTERY	0x10	Bit 4 must be 0
OVER_TEMPERATURE	0x20	Bit 5 must be 0
RADIO_LINK	0x40	Bit 6 is non-blocking
FATAL_ACTUATOR	0x80	Bit 7 must be 0

MANDATORY_MASK	0x0F	Bits 0–3 are all required
INHIBITOR_MASK	0xB0	Bits 4, 5 and 7 block readiness

Design overview

- Use AND to test required and inhibitor bits; no bit-position loop is needed.
- Use XOR between the mandatory mask and the observed mandatory bits to identify missing required sensors.
- Use OR to combine fault masks and NOT to form the clear-mask test for inhibitors.
- Radio bit 6 is deliberately excluded from the inhibitor mask.

Structured pseudo-code

```
CONSTANT GYROSCOPE : INTEGER ← 0x01
CONSTANT ACCELEROMETER : INTEGER ← 0x02
CONSTANT BAROMETER : INTEGER ← 0x04
CONSTANT GPS : INTEGER ← 0x08
CONSTANT LOW_BATTERY : INTEGER ← 0x10
CONSTANT OVER_TEMPERATURE : INTEGER ← 0x20
CONSTANT RADIO_LINK : INTEGER ← 0x40
CONSTANT FATAL_ACTUATOR : INTEGER ← 0x80
CONSTANT MANDATORY_MASK : INTEGER ← 0x0F
CONSTANT INHIBITOR_MASK : INTEGER ← 0xB0
CONSTANT BYTE_MASK : INTEGER ← 0xFF

METHOD CheckFlightReadiness(statusRegister : INTEGER)
  DECLARE presentMandatory, missingSensors, activeFaults : INTEGER
  DECLARE ready : BOOLEAN

  presentMandatory ← statusRegister AND MANDATORY_MASK
  missingSensors ← MANDATORY_MASK XOR presentMandatory
  activeFaults ← statusRegister AND INHIBITOR_MASK
  ready ← TRUE

  IF missingSensors <> 0 THEN
    ready ← FALSE
    DISPLAY "Missing sensor mask: ", HEX(missingSensors)
  ENDIF

  IF (activeFaults AND LOW_BATTERY) <> 0 THEN
    ready ← FALSE
    DISPLAY "ERROR: Low battery warning"
  ENDIF
  IF (activeFaults AND OVER_TEMPERATURE) <> 0 THEN
    ready ← FALSE
    DISPLAY "ERROR: Over-temperature warning"
  ENDIF
  IF (activeFaults AND FATAL_ACTUATOR) <> 0 THEN
    ready ← FALSE
    DISPLAY "ERROR: Fatal actuator hardware failure"
  ENDIF

  IF ready = TRUE THEN
    DISPLAY "Flight Ready"
  ELSE
    DISPLAY "Flight NOT Ready"
  ENDIF
END METHOD
```

NOT is represented by the complement operation when a clear-mask check is desired: (statusRegister AND (NOT INHIBITOR_MASK AND 0xFF)) = active non-inhibitor bits. The direct inhibitor test above is simpler and avoids signed-width ambiguity.

Trace: statusRegister = 0x4F

Operation	Working	Result
Mandatory bits	0x4F AND 0x0F	0x0F (all present)
Missing mask	0x0F XOR 0x0F	0x00
Inhibitors	0x4F AND 0xB0	0x00
Radio link	0x4F AND 0x40	0x40 (active, non-blocking)
Decision	No missing sensors or inhibitor faults	Flight Ready

Trace: statusRegister = 0x3B

Operation	Working	Result
Mandatory bits	0x3B AND 0x0F	0x0B
Missing mask	0x0F XOR 0x0B	0x04 (barometer missing)
Inhibitors	0x3B AND 0xB0	0x30
Low battery	0x30 AND 0x10	0x10 (set)
Over-temperature	0x30 AND 0x20	0x20 (set)
Fatal actuator	0x30 AND 0x80	0x00 (not set)
Decision	Missing barometer and two inhibitor faults	Flight NOT Ready

Expected diagnostics for 0x3B: Missing sensor mask 0x04; Low battery warning; Over-temperature warning; Flight NOT Ready.

Question 4: Optimized Divisor and Classification Engine

Design overview

- Reject non-positive input and repeat until a valid positive integer is entered.
- For N = 1, the proper-divisor sum is 0.
- For N greater than 1, begin with divisor 1 and test candidates only up to floor(square root of N).
- When d divides N, add d and its paired divisor N/d. Add the pair only when it differs from d to avoid double-counting a square root.
- CONTINUE skips non-divisors. Classification compares the proper-divisor sum with N.

Structured pseudo-code

```
BEGIN DivisorAndPrimeEngine
  DECLARE N, d, pairedDivisor, sumOfDivisors : INTEGER
  DECLARE validInput : BOOLEAN

  validInput ← FALSE
  WHILE validInput = FALSE DO
    INPUT N
    IF N <= 0 THEN
      DISPLAY "Invalid input. Enter a positive integer."
    ELSE
      validInput ← TRUE
    ENDIF
  ENDWHILE

  IF N = 1 THEN
    sumOfDivisors ← 0
  ELSE
    sumOfDivisors ← 1
    FOR d ← 2 TO FLOOR(SQRT(N)) STEP 1 DO
      IF N MOD d <> 0 THEN
        CONTINUE
      ENDIF
```

```
            pairedDivisor ← N / d
            sumOfDivisors ← sumOfDivisors + d
            IF pairedDivisor <> d THEN
               sumOfDivisors ← sumOfDivisors + pairedDivisor
            ENDIF
         ENDFOR
      ENDIF

      DISPLAY "Sum of proper divisors: ", sumOfDivisors
      IF sumOfDivisors = N THEN
         DISPLAY "Perfect Number"
      ELSE IF sumOfDivisors > N THEN
         DISPLAY "Abundant Number"
      ELSE
         DISPLAY "Deficient Number"
      ENDIF
   END DivisorAndPrimeEngine
```

Complexity: O(√N) divisor checks and O(1) extra space.

Trace for N = 28

d	28 MOD d	Paired divisor	Sum after step
Start	—	—	1
2	0	14	1 + 2 + 14 = 17
3	1 (CONTINUE)	—	17
4	0	7	17 + 4 + 7 = 28
5	1 (CONTINUE)	—	28

Result: sum = 28, therefore 28 is a Perfect Number.

Trace for N = 12

d	12 MOD d	Paired divisor	Sum after step
Start	—	—	1
2	0	6	1 + 2 + 6 = 9
3	0	4	9 + 3 + 4 = 16

Result: sum = 16, therefore 12 is an Abundant Number.

Question 5: State-Machine ATM Controller with Sentinel Loops

Design overview

- Authenticate with preset PIN 4821 and allow at most three attempts.
- If all attempts fail, lock the card and terminate before showing the menu.
- After successful authentication, use a sentinel-controlled menu loop; option 4 ends the session.
- Deposits must be positive. Withdrawals must be positive, within the available balance, and divisible by 100.

Sample assumption for the trace: starting balance = $1,000. The question does not specify an opening balance or a complete sequence of menu choices.

Structured pseudo-code

```
   BEGIN ATM_SessionController
      DECLARE presetPIN, enteredPIN, attempts, choice : INTEGER
      DECLARE balance, amount : REAL
      DECLARE authenticated, sessionActive : BOOLEAN

      presetPIN ← 4821
```

```
balance ← 1000
attempts ← 0
authenticated ← FALSE

WHILE attempts < 3 AND authenticated = FALSE DO
   INPUT enteredPIN
   attempts ← attempts + 1
   IF enteredPIN = presetPIN THEN
      authenticated ← TRUE
   ELSE
      DISPLAY "Incorrect PIN"
   ENDIF
ENDWHILE

IF authenticated = FALSE THEN
   DISPLAY "Card locked"
   STOP
ENDIF

sessionActive ← TRUE
WHILE sessionActive = TRUE DO
   DISPLAY "1 Balance  2 Deposit  3 Withdraw  4 Exit"
   INPUT choice

   SWITCH choice
      CASE 1:
         DISPLAY "Available balance: ", balance
         BREAK

      CASE 2:
         INPUT amount
         IF amount > 0 THEN
            balance ← balance + amount
            DISPLAY "Deposit successful"
         ELSE
            DISPLAY "Invalid deposit amount"
         ENDIF
         BREAK

      CASE 3:
         INPUT amount
         IF amount <= 0 THEN
            DISPLAY "Invalid withdrawal amount"
         ELSE IF amount > balance THEN
            DISPLAY "Insufficient balance"
         ELSE IF amount MOD 100 <> 0 THEN
            DISPLAY "Withdrawal must be a multiple of 100"
         ELSE
            balance ← balance - amount
            DISPLAY "Withdrawal successful"
            DISPLAY "Available balance: ", balance
         ENDIF
         BREAK

      CASE 4:
         sessionActive ← FALSE
         DISPLAY "Thank you. Session ended."
         BREAK

      DEFAULT:
         DISPLAY "Invalid menu option"
   ENDSWITCH
```

```
    ENDWHILE
END ATM_SessionController
```

Dry-run test trace

Step	Input / action	System response	Balance
1	PIN = 1234	Incorrect PIN; attempts = 1	$1,000
2	PIN = 4821	PIN accepted; authentication successful	$1,000
3	Menu choice = 3; amount = $250	Rejected: not a multiple of 100	$1,000
4	Menu choice = 3; amount = $300	Withdrawal successful	$700
5	Menu choice = 4	Thank you. Session ended.	$700

Expected key outputs, in order: Incorrect PIN; Withdrawal must be a multiple of 100; Withdrawal successful; Available balance: 700; Thank you. Session ended.

Quick Revision

Debugging – important rules

Concept	Remember
Operator precedence	Use parentheses around bitwise tests: (x & MASK) == MASK.
Unsigned loop	Unsigned values wrap around; do not use i >= 0 as a countdown condition.
Scope resolution	::name accesses a global namespace-scope name, not an enclosing local variable.
switch	Use integral/enum selector and integral constant case labels; add break to prevent fall-through.
Case variable scope	Put declarations inside braces for each case block.
References	A reference must be initialized; non-const lvalue references cannot bind to temporary values.
Const reference	Read-only; can bind to a temporary and extend its lifetime.
Dynamic arrays	new[] must pair with delete[]; new pairs with delete.
Stray semicolon	A semicolon after for/while creates an empty loop body.

Pseudo-code – important patterns

Task	Core pattern
Class design	PRIVATE attributes; PUBLIC constructor and methods; validate before calculating.
Tiered billing	Calculate each tier separately, add subtotals, then apply surcharge as specified.
Tax brackets	Tax only the income portion in each bracket; cap deductions before subtraction.
Bit masks	AND tests bits; OR combines masks; XOR identifies differences; NOT complements bits.
Divisors	Check d up to √N and add paired divisor N/d without double-counting squares.
ATM authentication	Count attempts; stop after three failures; only then enter the session loop.
Sentinel loop	Continue until the explicit exit value is selected.

Verified numerical answers

Question	Final result
A1	Valid Odd Number: 15; countdown: 3 2 1 0
A2	Inner total = 1120; global total = 1180; local total = 1050
A3	Calculated Value = 100
A4	Reference outputs: 25 25; 100; 3; 50
A5	Array total = 150
B1	370 units; subtotal $930; surcharge $93; final $1,023
B2	Taxable income $1,025,000; tax + cess $124,800
B3	0x4F ready; 0x3B not ready (missing 0x04; faults 0x10 and 0x20)
B4	28 perfect; 12 abundant
B5	Starting $1,000; $250 rejected; $300 withdrawal succeeds; ending $700